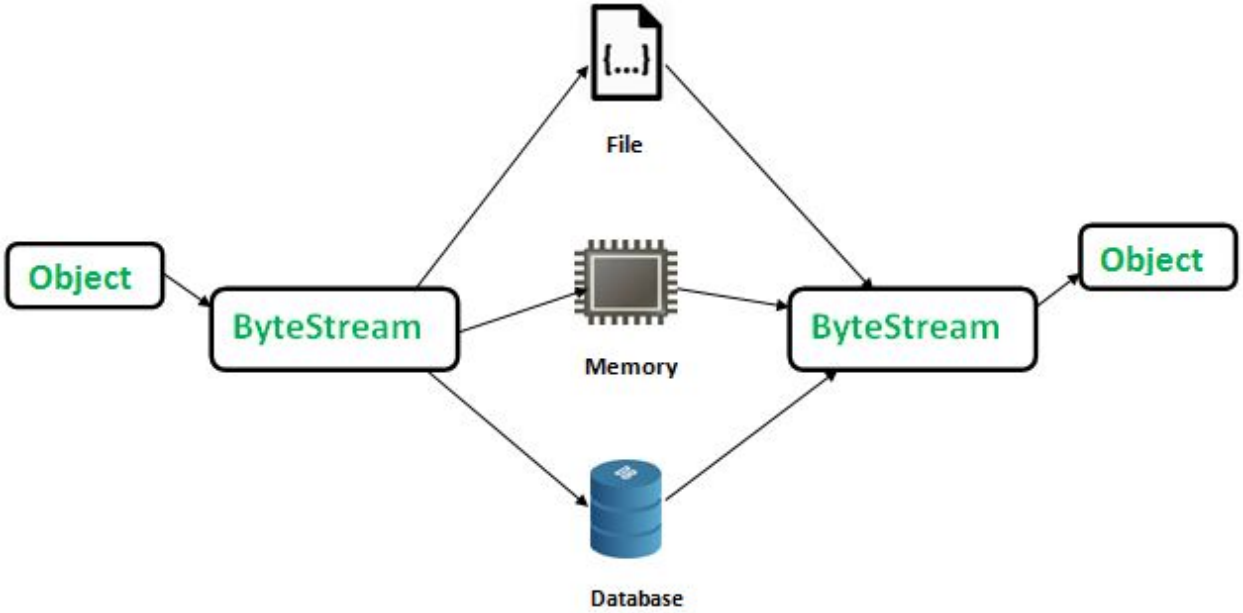
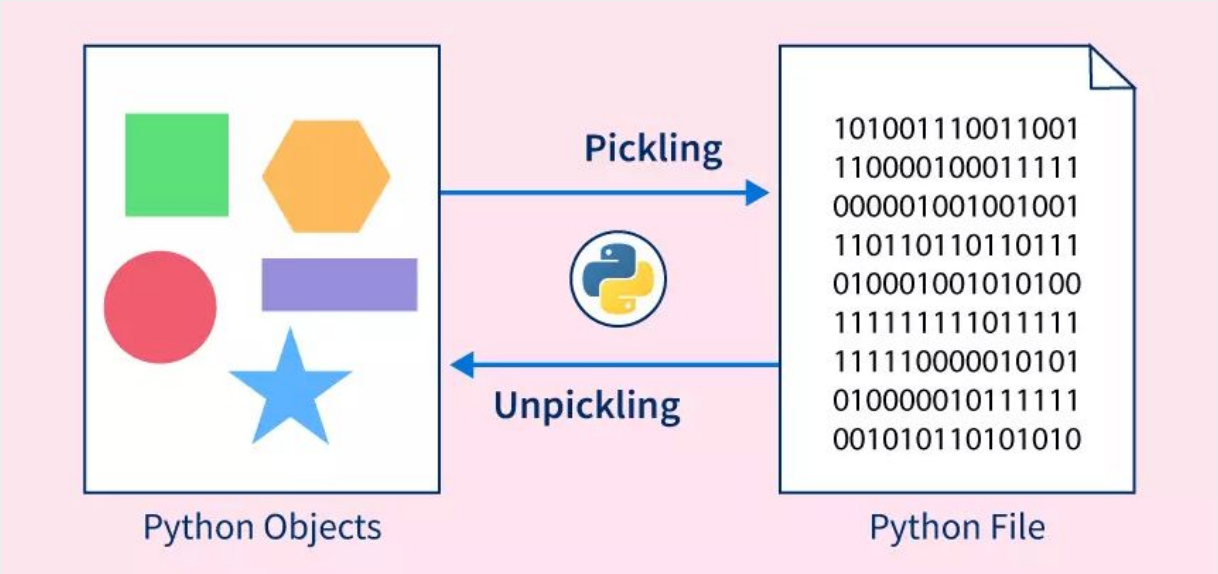
Pickle y JSON

Serialización

Serialización y deserialización



Pickle


Pickling
Unpickling
101001110011001
110000100011111
000001001001001
110110110110111
010001001010100
111111111011111
111110000010101
010000010111111
001010110101010
Python Objects
Python File

Pickle

Ejemplo de serialización con protocolo pickle

```python
import pickle

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person(name={self.name},
age={self.age})"

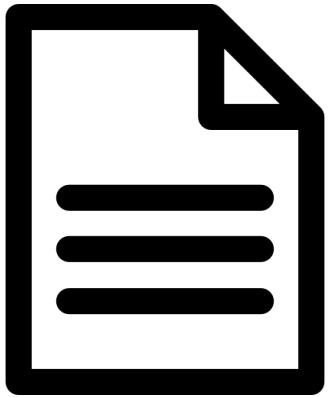
    def __str__(self):
        return f"Person: {self.name}, Age: {self.age}"

    def __eq__(self, other):
        if isinstance(other, Person):
            return self.name == other.name and self.age
== other.age
        return False

nahuel = Person("Nahuel", 22)

with open("nahuel.pkl", "wb") as f:
    pickle.dump(nahuel, f)
```

¿Qué es lo que hizo este código?



nahuel.pkl



```
8004953700000000
000008C085F5F6D6
1696E5F5F948C0650
6572736F6E9493942
981947D94288C046
E616D65948C064E6
16875656C948C036
16765944B1675622E
```

Pickle

Ejemplo de deserialización con protocolo pickle

```python
import pickle

class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person(name={self.name}, age={self.age})"

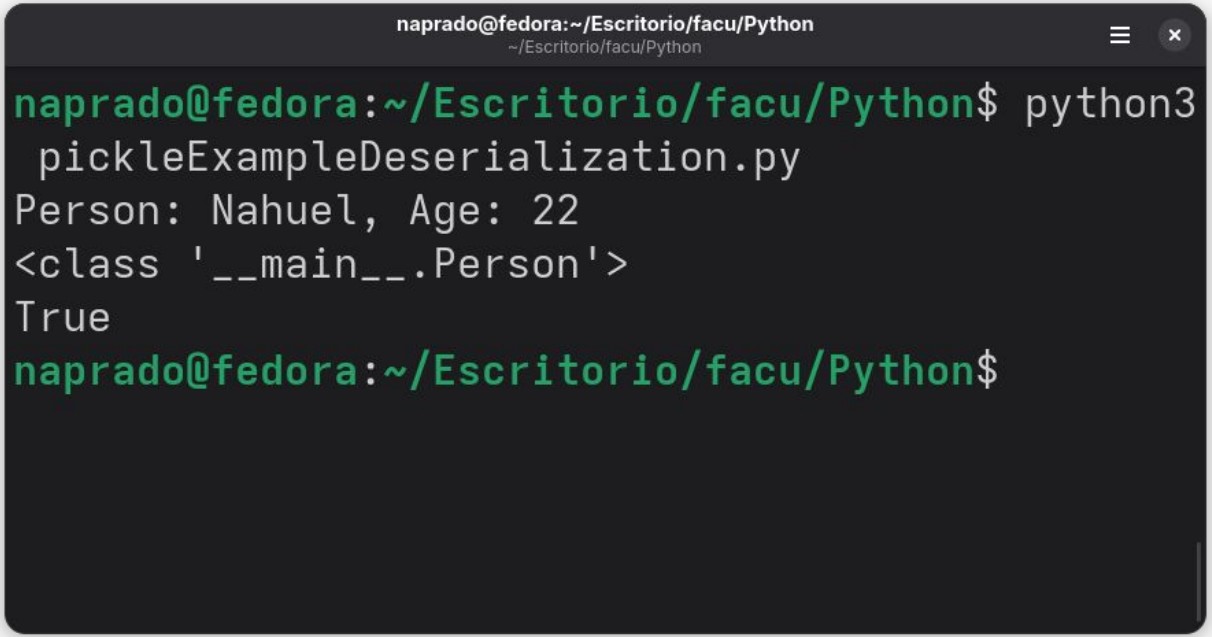
    def __str__(self):
        return f"Person: {self.name}, Age: {self.age}"

    def __eq__(self, other):
        if isinstance(other, Person):
            return self.name == other.name and self.age == other.age
        return False

with open("nahuel.pkl", "rb") as f:
    persona_cargada = pickle.load(f)

print(persona_cargada)
print(type(persona_cargada))
print(persona_cargada == Person("Nahuel", 22))
```

¿Cual es la salida?



Métodos de Pickle

pickle.dump(objeto, archivo)

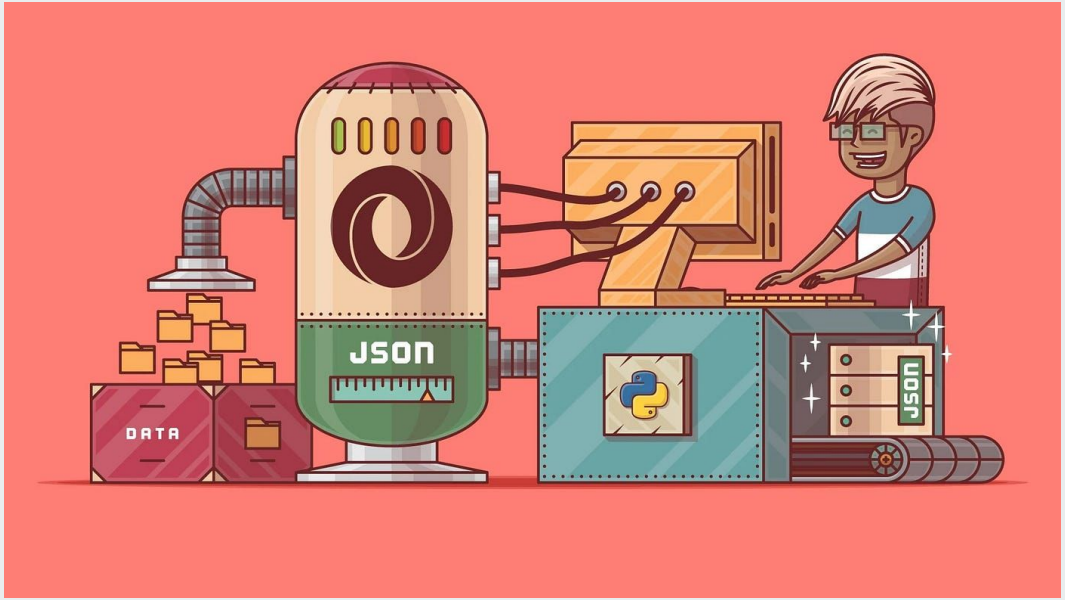
objeto = pickle.load(archivo)

data = pickle.dumps(objeto)

objeto = pickle.loads(data)

JSON

JavaScript Object Notation



Equivalencias entre objetos de python y JSONs

Python	JSON
dict	objeto { }
list,tuple	array [ ]
str	string “ “
int, float	number
True,False	true,false
None	null

Analicemos el siguiente JSON

```
{
 "nombre": "Nahuel",
 "edad": 22,
 "altura": 1.73,
 "estudia": true,
 "hobbies": ["programar", "música"],
 "direccion": {
   "calle": "Av. Siempreviva",
   "numero": 742,
   "departamento": null
 },
 "historial": [
   {"año": 2024, "cuatrimestre": 1,"evento":
"Ayudante Introducción a la informática"},
   {"año": 2024, "cuatrimestre": 2,"evento":
"Ayudante Introducción a la informática"},
   {"año": 2025, "cuatrimestre": 1,"evento":
"Ayudante Estructuras de datos y
programación"}
 ]
}
```

Método to_dict()

Buscamos guardar un json que recordemos son diccionarios

```python
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def __repr__(self):
        return f"Person(name={self.name}, age={self.age})"
    def __str__(self):
        return f"Person: {self.name}, Age: {self.age}"
    def __eq__(self, other):
        if isinstance(other, Person):
            return self.name == other.name and self.age == other.age
        return False

    def to_dict(self):
        return {
            "name": self.name,
            "age": self.age
        }
```

JSON

Ejemplo de serialización con archivos JSON

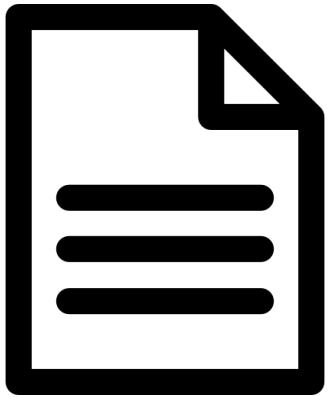
```python
import json

# Here goes the Person Class

nahuel = Person("Nahuel", 22)

with open("nahuel.json", "w") as f:
    json.dump(nahuel.to_dict(), f, indent=4)
```

¿Qué es lo que hizo este código?





```
{
    "name": "Nahuel",
    "age": 22
}
```

nahuel.json

JSON

Ejemplo de deserialización con archivos JSON

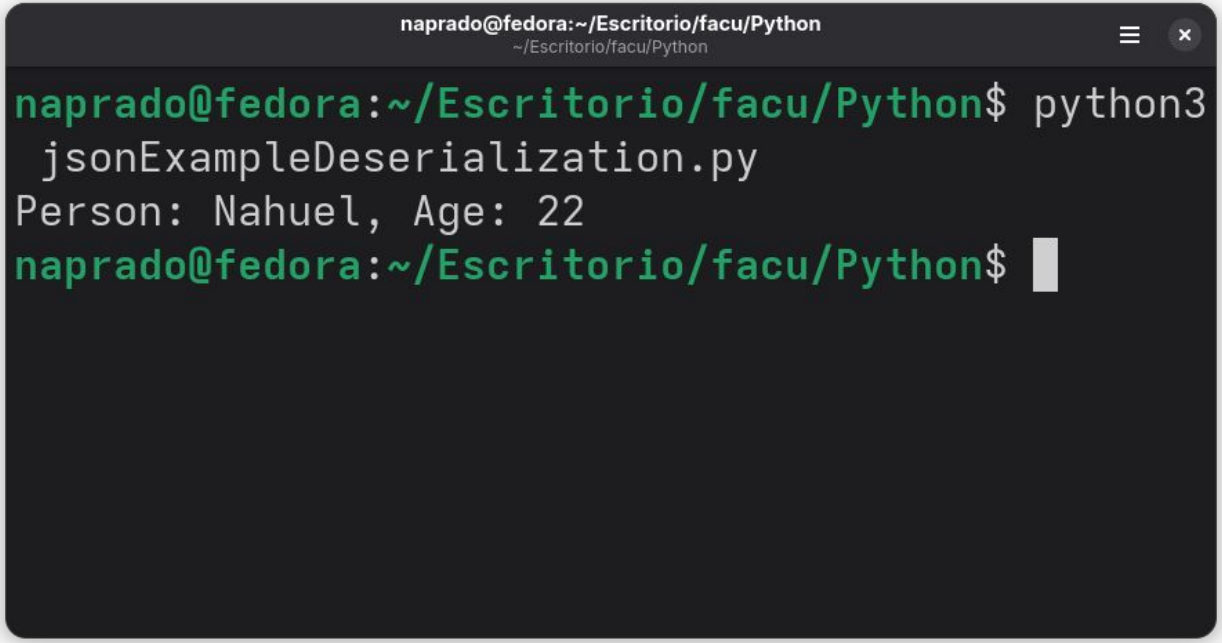
```python
import json

# Here goes the Person Class

with open("nahuel.json", "r") as f:
    data = json.load(f)
    persona_cargada = Person(**data)

print(persona_cargada)
```

¿Cual es la salida?



Métodos de JSON

json.dump(diccionario, archivo, indent=4)

diccionario= json.load(archivo)

string = json.dumps(diccionario, indent=4)

diccionario =json.loads(JSON_como_string)

Diferencias entre JSON y pickle

	JSON	pickle
Es legible por humanos	Si	No
Es exclusivo de Python	No	Si
Soporta datos complejos (clases propias)	Se pueden implementar soluciones pero no es directo	Si

¿Consultas?